PA3 Report: Blocking the Outer Product

Name: Alexander Stanton · **Date:** 2026-10-18 · **Machine:** eel (i9-13900K, E-cores only, Turbo off)

Starting Point

The kernel is OPQ's `c[i+j] += a[i] * b[j]` over the whole `(i, j)` rectangle, now cut into tiles. Part A makes one tile run as fast as possible on one core; Part B spreads the tiles over the 16 E-cores. All headline timing is at **100,000 × 100,000** (10^{10} multiply-adds, $2 \cdot 10^{10}$ flops). Peak (TPP) is 35.2 GFLOP/s per E-core, 563.2 GFLOP/s for all 16. Sequential runs are pinned to core 16; parallel runs use `_gate.sh`'s binding (cores 16–31, `OMP_PROC_BIND=close`, cluster-cyclic places: the first four threads land on four different clusters). Every variant passed the shared gate (42 oracle cases and 14 gold cases, sequential and parallel checking builds) before it was timed, except the one deliberately racy diagnostic in Experiment 11. Timings are medians of 5 or 7 reps from `sweep.sh`; hardware counters are from `perf stat` on the `cpu_atom` (E-core) events, normalized per useful vector FMA.

Nick's PA2 report was used to direct early exploration of some of our PA2 implementation deficiencies and and as a stylistic reference for this report.

▶ Background: what an FMA is, and why everything is counted per FMA

▶ Background: tiles, and the 4-loop structure

REFERENCE POINTS, 100K × 100K, ONE CORE	GFLOP/S	% TPP
OPQ (PA2 measurements, not re-run here: 26.5% at 160K)	≈ 9.4	≈ 27
BKQ, plain blocking, best block (Experiment 1)	13.22	37.6
BKQ7, final single-core kernel, best block (Experiment 8)	25.40	72.2

The kernel has to go through the shared `BPM_ACC` macro for every addition into `c`: a plain `+=` in the timed build (compiled with `-ffast-math`), Kahan-compensated in the checking build. That rules out writing the hot loop with intrinsics, which comes up twice below.

Experiment 1: plain blocking (BKQ) `sequential`

QUESTION

Does tiling OPQ's two loops, with nothing else changed, change the single-core rate, and how does it depend on the block size?

HYPOTHESIS

At PA2's small sizes BKQ matches OPQ (~37% TPP): the inner loop, and so its 2 loads and 1 store per vector FMA, is unchanged. At large sizes, where OPQ's rows outgrow L1 and it falls to ~27%, blocking should hold the small-size plateau.

EXPERIMENT DESIGN

Square tiles, 4 loops, edge tiles incomplete, runtime knob `--tune block_size` (64 when absent). The inner `j` loop vectorizes 8-wide exactly as OPQ's (`vbroadcastss` of `a[i]`, `vfmadd213ps` with `c` as a memory operand, one store). Block sweeps: 16–1,024 at 1,000 × 4,000 and 8,000 × 2,000 (5 reps), then 256–4,096 at 20K, 40K, 60K, 80K and 100K square (7 reps).

EXPERIMENT RUN

`03-BKQ/results/2026.10.18.00{1..7}_bs_sweep_*.csv`.

ANALYSIS

BLOCK	16	64	128	256	512	1024	2048	4096
8,000 × 2,000	7.7%	25.3%	35.9%	37.2%	37.8%	36.1%	–	–
100K × 100K	–	–	–	37.2%	38.0%	36.4%	37.2%	36.4%

- From 192 up, BKQ sits at 36–38% TPP at every block size and every problem size from 1,000 × 4,000 to 100K × 100K, within 2–4% of OPQ's small-size rate.
- At large sizes that is ~1.4× OPQ's ~27%: blocking keeps the inner loop's data in cache where OPQ's long rows don't.
- Small blocks are much worse than "a little loop overhead": bs = 16 runs at a fifth of OPQ, and the rate climbs roughly in proportion to bs up to ~96. Not explained (each 16-long row is two vector iterations plus loop setup).

CONCLUSIONS

Supported. Blocking by itself fixes the large-size cache loss but not the per-FMA load limit: 2 loads and 1 store per vector FMA (6 µops) hold the kernel at ~37% at any size.

Experiment 2: unroll-and-jam, R = 8 (BKQ2) `sequential`

QUESTION

The limit is loads per FMA. Unroll-and-jam (experiment 6, step 1 of Nick's PA2 report) loads and stores each vector of `c` once per 8 FMAs instead of once per FMA. How much of that shows up inside a tile?

HYPOTHESIS

Loads per FMA fall from 2 to 9/8 and BKQ2 rises to ~55–60% TPP. The assembly showed that `-ffast-math` regroups the 8 products into pairs (4 multiplies + 4 FMAs + 4 adds, 12 FP instructions for 8 FMAs of work), which caps the loop at 8/12 = 67%.

▶ Background: unroll-and-jam on the outer product

EXPERIMENT DESIGN

BKQ's tiling, R = 8 fixed, 8 values of `a` hoisted into locals, ragged ends and leftover rows as plain rows. Same large-size block sweeps as Experiment 1 (BKQ re-swept alongside for a direct comparison).

EXPERIMENT RUN

`03-BKQ2/results/2026.10.18.00{3..7}_bs_sweep_*.csv`; first small-size sweeps in `.001` / `.002`.

ANALYSIS

BLOCK, 100K	256	512	1024	2048	3072	4096
BKQ	37.2%	38.0%	36.4%	37.2%	37.5%	36.4%
BKQ2	41.2%	48.6%	53.4%	55.7%	56.5%	56.8%
BKQ2 / BKQ	1.11	1.28	1.47	1.50	1.51	1.56

- BKQ2 reaches 56.8% (20.0 GFLOP/s) at bs = 4,096, 1.56× BKQ, the same at 40K–100K (20K up to 1 GFLOP/s lower).

- It rises with block size all the way to 4,096. A footprint estimate (≈ 12·bs bytes per tile) predicted a drop past bs ≈ 2,700 when a tile outgrows L1; it did not happen.
- Small blocks lose: below bs = 192 BKQ2 is slower than BKQ, because the fused loop covers only bs − 7 of each row.

> **Deviation.** The first BKQ2 sweep at 1,000 × 4,000 (≈ 1 ms per rep) was unusable for bs = 128–768: within each block size the 5 reps got steadily faster (bs = 384: 0.710 → 0.542 ms) while another user's rsync was running. That is why all later single-core work is at 20K–100K, where a rep takes about a second; rep spreads there were within ~0.5%.

CONCLUSIONS

Supported: 56.8%, within the predicted range and under the 67% regrouping cap. The best block is not bracketed.

Experiment 3: keep the chain of 8 FMAs (BKQ3) `sequential`

QUESTION

Does removing the 12-instruction regrouping (experiment 6, step 2 of Nick's PA2 report) lift the kernel toward the 67% cap and past it?

HYPOTHESIS

With `__attribute__((optimize("no-associative-math")))` on the kernel only, the loop becomes one load of `b`, 8 chained FMAs and one store. BKQ3 beats BKQ2 and levels off at 60–67%, where the chain of 8 dependent FMAs (6-cycle latency) becomes the next limit.

EXPERIMENT DESIGN

BKQ2 plus the attribute; the rest of the build keeps `-ffast-math`. Same block sweeps. The hot loop was checked in the disassembly:

```
vmovups     (%r14,%rax,1),%ymm0          ; b window, row 0
vfmadd213ps (%rdx,%rax,1),%ymm15,%ymm0   ; * a0, + c
vfmadd231ps (%r11,%rax,1),%ymm14,%ymm0   ; + a1 * b window, row 1
...                                       ; 5 more
vfmadd231ps (%rbx,%rax,1),%ymm8,%ymm0    ; + a7 * b window, row 7
vmovups     %ymm0,(%rdx,%rax,1)          ; store c
```

EXPERIMENT RUN

`03-BKQ3/results/2026.10.18.00{1..5}_bs_sweep_*.csv`. Every rep within 0.3% of its median.

ANALYSIS

BKQ3 is 4–6% faster than BKQ2 at every block size and problem size; best 59.1% (20.8 GFLOP/s) at 3,072–4,096. That is far less than the ~16% this step gave in Nick's PA2 report, and BKQ2 was already well under its cap, so regrouping was not the main limit.

FOLLOW-UP: WHAT HOLDS BKQ3 AT 59%?

Four counter groups, each small enough to be counted 100% of the time, at 100K, bs = 3,072:

BKQ3, PER VECTOR FMA	VALUE
cycles (1.0 = TPP)	1.687
load µops	2.303 (9 loads per 8 FMAs, 2 µops each)
split loads	0.438 (19% of load µops)
L1 misses	0.10% of loads
load buffer full	28.4% of cycles
memory scheduler full / FP scheduler full	59.3% / 0.01% of slots

▶ Background: split loads

CONCLUSIONS

Partly supported: the chain helps by ~5%, but the next limit is not latency (the FP scheduler is never full). BKQ3 is load-side bound: the load buffer is full 28% of cycles and one load µop in five is a split. That points to aligned copies of `b` (from experiment 6 of Nick's PA2 report) rather than interleaved accumulators.

Experiment 4: four aligned, shifted copies of b (BKQ4) `sequential`

QUESTION

Does reading `b` from aligned copies remove the split loads, and what does that buy?

HYPOTHESIS

Split loads of `b` fall to near zero (only the unaligned loads of `c`, 1 per 8 FMAs, may still split) and BKQ4 rises above 59%. The copies add footprint, so the best block may move down.

EXPERIMENT DESIGN

Once per call, inside the timed kernel, build 4 copies of `b` on 64-byte-aligned bases, copy `s` shifted right by `s` floats (`bc[s][x+s] = b[x]`). For each tile, row `r` reads from the copy that puts all its loads on 16-byte boundaries, which never cross a line. Only the fused loop uses the copies. Same block sweeps.

EXPERIMENT RUN

`03-BKQ4/results/2026.10.18.00{1..5}_bs_sweep_*.csv`, then the same four counter groups at 100K/3,072, 80K/3,072 and 100K/1,536.

ANALYSIS

BLOCK	1024	1536	2048	3072	4096
BKQ4 at 40K (BKQ4 / BKQ3)	58.3% (1.05)	59.1% (1.04)	59.9% (1.03)	61.6% (1.04)	62.8% (1.06)
BKQ4 at 100K (BKQ4 / BKQ3)	58.9% (1.06)	60.3% (1.06)	53.8% (0.92)	55.3% (0.94)	55.9% (0.95)

PER VECTOR FMA	BKQ3 100K/3072	BKQ4 100K/3072	BKQ4 80K/3072	BKQ4 100K/1536
cycles	1.687	1.806	1.627	1.654
split loads	0.438	0.064	0.064	0.065
L1 misses, % of loads	0.10%	29.9%	9.6%	9.3%
load buffer full, % of cycles	28.4%	23.8%	15.9%	12.9%
memory / FP scheduler full, % of slots	59.3 / 0.0	61.5 / 1.0	29.4 / 28.8	23.9 / 30.4

- The copies removed 85% of the split loads at every configuration (0.438 → 0.064).
- At 20K–80K BKQ4 is 2–10% faster than BKQ3; best 62.8% (22.1 GFLOP/s) at 40K, bs = 4,096.
- At 100K with bs ≥ 2,048 it is 5–8% *slower* than BKQ3, with tight reps. The counters line up with it: 30% of loads miss L1 at 100K/3,072 against 10% at 80K/3,072 with the same block.

- L1 misses appeared at all (0.1% → ~10%) once the copies were added, and with the splits gone the FP scheduler is now full ~30% of slots: the FMA chain shows up as a limit.

CONCLUSIONS

Supported for the splits, but the copies brought L1 misses, and something about 100K with large blocks triples them. The copies were placed back to back, `dB + 4` floats apart rounded to a line, so where they sit relative to each other changes with `dB`.

Experiment 5: where the copies sit (BKQ4, copy spacing) `sequential`

QUESTION

Are the extra misses at 100K a capacity problem (the tile no longer fits) or a placement problem (the copies collide in the same L1 sets)?

HYPOTHESIS

Placement. Moving the copies apart with padding brings some spacings back to ~61–62% TPP and leaves others near 55%. If the misses were capacity misses, padding wouldn't change them.

▶ Background: L1 sets, and why 4,096 bytes

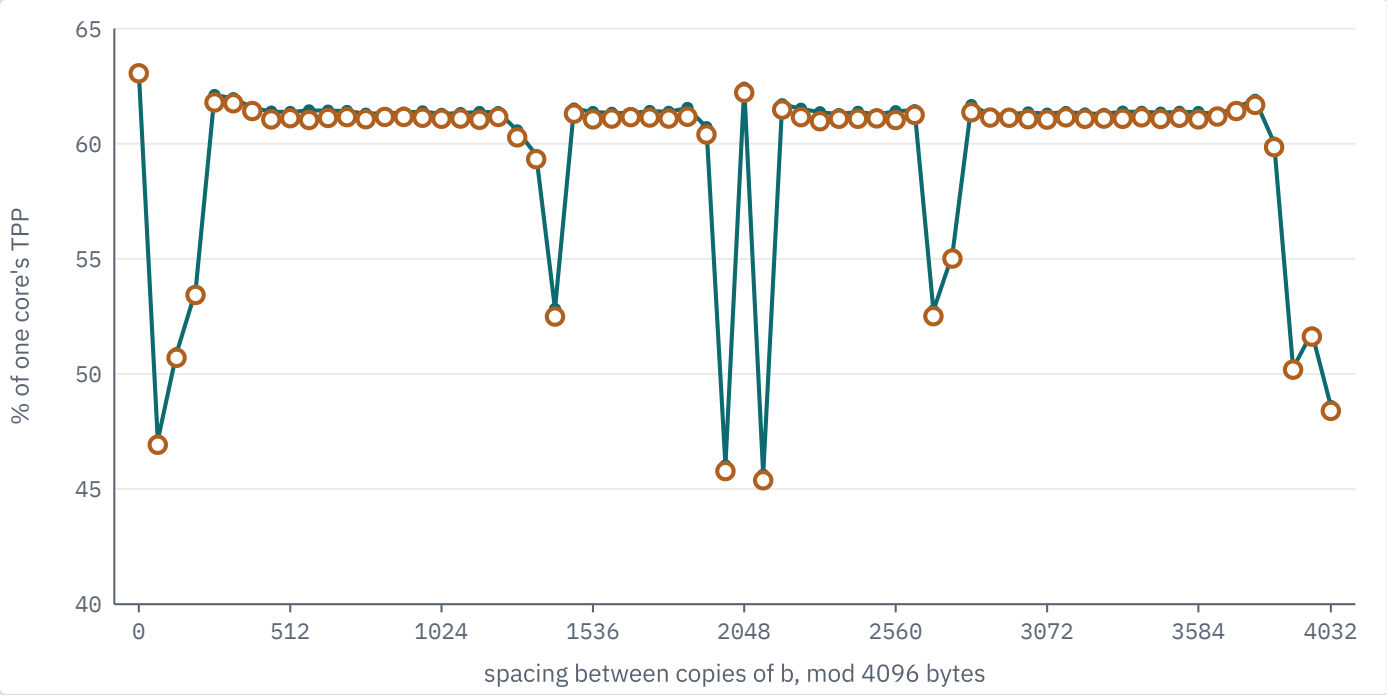
EXPERIMENT DESIGN

Runtime knob `--tune copy_pad=N`: N floats (rounded up to 16) between consecutive copies; only addresses change. 64 pads one line apart cover every spacing mod 4,096 bytes. Swept at 100K, 80K and 40K, bs = 3,072, 5 reps; one perf run per pad at 0, 16, 336 and 864 (100K).

EXPERIMENT RUN

`03-BKQ4/results/2026.10.18.00{6,7,8}_copy_pad_sweep_*.csv`. Reps within 2%.

ANALYSIS



- The rate depends on the copy spacing mod 4,096 alone: at every spacing the three sizes agree within ~0.3 points. Most spacings give 61–62%; the default at 100K (2,752) happens to be a bad one, which is all the 100K drop of Experiment 4 was.
- Bad spacings sit just next to, not at, multiples of 4,096 and of 2,048 and 4,096/3. Read off the table: a spacing `S` is bad when `S`, `2S` or `3S` mod 4,096 (the distances between pairs of copies) lands 64–192 bytes from a multiple of 4,096 without being one. The worst spacings cost 28% (45.6% vs 63.1%).
- Exactly 0 mod 4,096 is the best spacing at every size: 63.1% (22.2 GFLOP/s).
- `ld_blocks.address_alias` equals the L1-miss count within 2–10% at all four profiled pads, and both vary 8× with padding. From these counters I can't tell L1 set conflicts from 4K aliasing against the stores to `c`.

CONCLUSIONS

Supported: the misses are a placement effect, not capacity. From here on (all BKP variants and BKQ5–7) the copy spacing is rounded up to a multiple of 4,096 bytes. The mechanism stays open.

Experiment 6: unrolling the fused loop (BKQ5) `sequential`

QUESTION

Once the aligned copies removed the split loads, BKQ4's profile showed the FP scheduler full ~30% of slots. Each output vector is a chain of 8 dependent FMAs (≈ 48 cycles). Does putting more independent chains next to each other help?

HYPOTHESIS

`#pragma GCC unroll 4` on the fused loop gives 4 independent chains per pass, the FP scheduler share falls, and the rate rises above ~63%. Interleaving the chains instruction by instruction would need intrinsics, which `BPM_ACC` rules out.

EXPERIMENT DESIGN

BKQ4 with the 4,096-byte spacing plus the pragma. In the disassembly the unroll works, but the 4 chains are emitted one after another, not interleaved. GCC's pre-allocation scheduler (`optimize("schedule-insns")`, tried on a scratch copy) interleaved two chains at a time but added a `vmovaps` register copy for nearly every FMA, so it was not used. Quick check: 100K, core 16, 3 reps, twice; control = the same source without the pragma.

ANALYSIS

BLOCK	BKQ5 (UNROLL 4)	CONTROL (NO PRAGMA)	BKP4 KERNEL, 1 THREAD
1024	20.71	20.74	20.72
2048	18.17	–	21.12
3072	17.37	22.20	22.25

Neutral at 1,024 and 14–22% slower at larger blocks; the two runs matched within 0.1%. The control matches the unchanged kernel, so the loss comes from the unroll itself. Not profiled, not explained.

CONCLUSIONS

Refuted, and set aside: plain-C unrolling doesn't interleave the chains, and costs speed at large blocks.

Experiment 7: rectangular tiles (BKQ6) sequential

QUESTION

BKQ4 still missed L1 on ~10% of loads even at a good placement. A tile that is tall in `i` and narrow in `j` touches a narrower window of the copies of `b`. Does any width below the square

tile do better?

HYPOTHESIS

With the height fixed, the rate first rises with width (the scalar ragged ends are 56 of 8w products per group of 8 rows) and, if the copy window's L1 misses matter, peaks below the square tile.

EXPERIMENT DESIGN

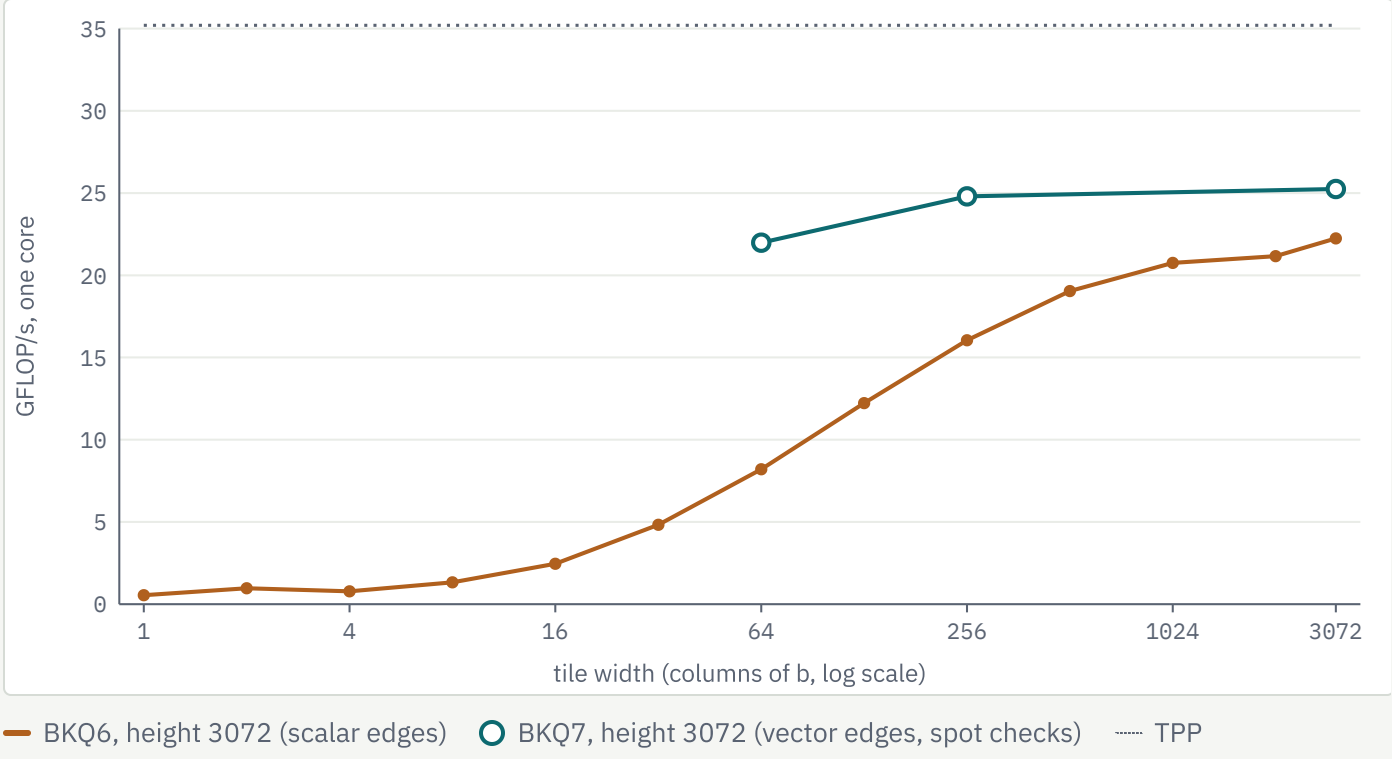
New knob `block_width` (default = `block_size`); everything else is the BKQ4 kernel with 4,096-byte spacing. Widths 1 to n at heights n = 3,072 and 1,024, 100K, 3 reps.

> **Deviation.** The shared gate tune (`BPM_GATE_TUNE` in `shared/include/kernel.h`) has no `block_width` entry, so the gate only runs square 64 × 64 tiles. The header says to add every new key, but it is shared infrastructure and was not changed. Instead, scratch checking builds with `block_width` = 1, 5, 8, 24, 40, 64 added to the gate tune were run; all passed. The same was done for BKQ7.

EXPERIMENT RUN

`03-BKQ6/results/2026.10.18.00{2,3}_width_bs*_100000x100000.csv`. Reps within 0.3% except one slow rep at w = 1.

ANALYSIS



- The rate rises with width all the way to the square tile; no narrower tile beats it. Height makes no difference at all: at every width, heights 1,024 and 3,072 agree within 0.4%.
- A cost model fits the points from w = 32 up within ~4%: time per group of 8 rows = x · (vector iterations) + y · (scalar products: 56 ragged-end products plus 8 per leftover fused-loop iteration) + z. It puts one scalar product at ~0.25 of an 8-FMA vector iteration, about 16× the cost per flop, and z ≈ 0. At w = 64, a group does 64 scalar products against 56 vector FMAs.
- w = 4 is slower than w = 2 (0.79 vs 0.97 GFLOP/s); not explained.

CONCLUSIONS

Refuted: narrow tiles don't help. The narrow-width loss is the fixed scalar edge work of each row group, and by the model it is still ~10% of the time of a square 1,024 tile. That made it the next target.

Experiment 8: doing the edges in the vector loop (BKQ7) `sequential`

QUESTION

Can every product of a full row group go through the vectorized fused loop, and what does that buy?

HYPOTHESIS

Up to ~10% at bs = 1,024 (the model's scalar share), most of the narrow-tile penalty gone, at the cost of up to 2 extra vector iterations of zero products per group.

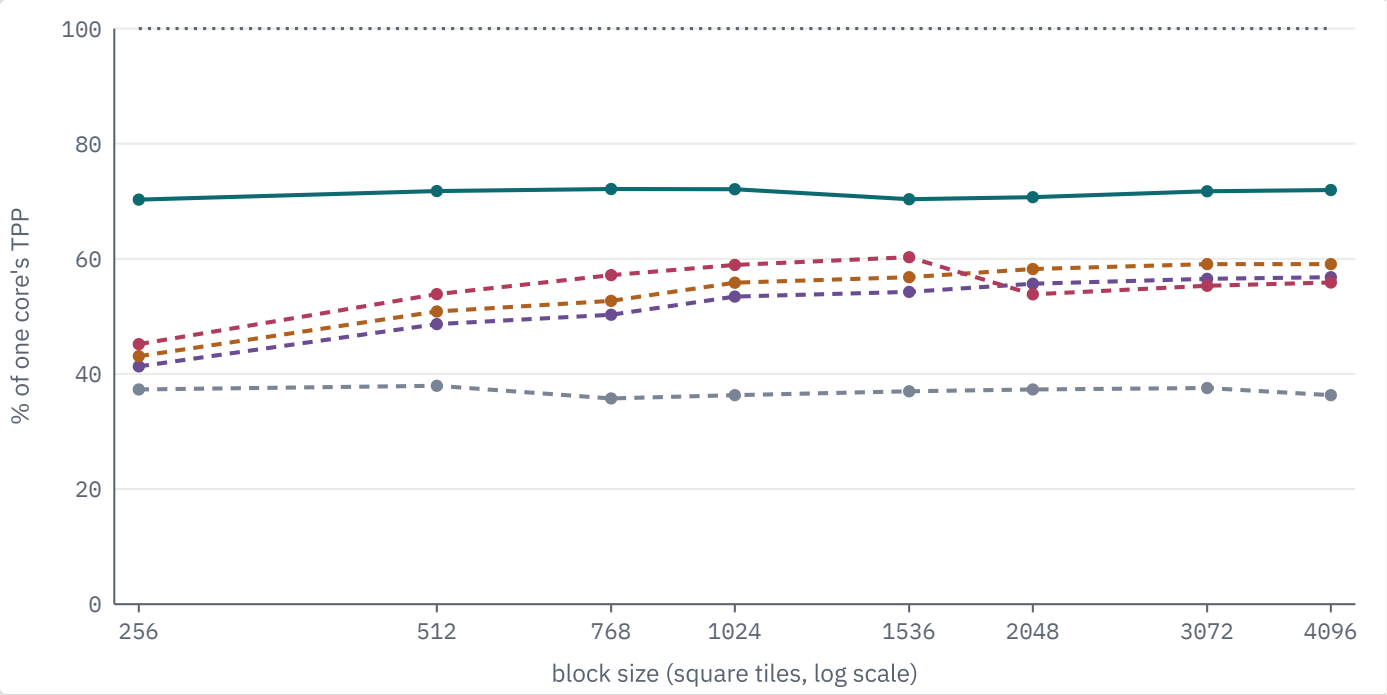
EXPERIMENT DESIGN

The copies of `b` become one zero-padded segment per tile column: in copy `s`, segment `J` holds the tile's `w` values of `b` after 7 zeros, then zeros. Row `r` reads through `p[r] = segment + s + 7 − r`, so `p[r][u] = b[jj + u − r]` inside the tile and **0 outside it**. The fused loop then runs over the whole trapezoid of the group (`u = 0 .. w+6`), rounded up to a multiple of 8 so the vector loop has no remainder. Where a row has no `b` in the tile it adds `a · 0`, which leaves `c` unchanged for finite inputs. Every segment starts on a 64-byte boundary, so the copy for row `r` is simply `(r + 1) mod 4`. Only the leftover rows of the bottom tile row (height not a multiple of 8) stay scalar. The hot loop is unchanged: one load of `b`, 8 chained FMAs, one store.

EXPERIMENT RUN

Block sweep 256–4,096 at 100K, 7 reps (`03-BKQ7/results/2026.10.18.002_bs_sweep_100000x100000.csv`); then every single-core variant re-timed at its best block (`*_best_bs*_100000x100000.csv` in each directory). Reps within 0.2%.

ANALYSIS



EACH VARIANT AT ITS BEST BLOCK, 100K, FRESH RUNS	GFLOP/S	% TPP
BKQ (3,072)	13.22	37.6
BKQ2 (4,096)	19.99	56.8
BKQ3 (4,096)	20.82	59.1
BKQ4 (1,536; dB-dependent spacing)	21.24	60.3
BKQ6 (3,072; BKQ4 with 4,096-byte spacing)	22.23	63.2
BKQ7 (768)	**25.40**	**72.2**

- BKQ7 is nearly flat across block sizes: 70.3–72.1% from 256 to 4,096, a 2.6% range; 768 and 1,024 tie (25.39 / 25.38). Even the untuned default block of 64 runs at 21.5 GFLOP/s, against 7.8 for the BKQ4 kernel (with 4,096-byte spacing) at 64.

- It beats BKQ4 at every block size, by 56% at 256 and 22% at 1,024. On narrow tiles the gain is larger still (3,072 × 64: 8.2 → 22.0 GFLOP/s).
- The 22% at 1,024 is about twice the model's estimate. Either scalar edge work costs more than the model allows, or BKQ7 gains something else as well (not identified).

CONCLUSIONS

Supported, and more than predicted: 72.2% of TPP on one core, and the block size no longer matters for single-core speed, which leaves it free to be chosen for the parallel version.

Experiment 9: tile rows with an array reduction (BKP1) `parallel`

QUESTION

How far does the simplest race-free split go: whole tile rows dealt out statically, each thread adding into its own copy of `c`?

HYPOTHESIS

Near-linear, limited by load imbalance. At 100K there are 98 / 66 / 33 tile rows for bs = 1,024 / 1,536 / 3,072; at 16 threads the busiest thread has 7 / 5 / 3 rows against an average of 6.1 / 4.1 / 2.1, capping speedup at ~14.0× / ~13.2× / ~11×.

▶ Background: array reductions

EXPERIMENT DESIGN

BKQ4 with 4,096-byte spacing; `#pragma omp parallel for schedule(static) reduction(+:c[:n])` on the `ii` loop (the checking build reduces the Kahan array `comp` the same way). Copies of `b` built before the parallel region and shared. Threads 1–16 at bs = 1,024, 1,536, 3,072, 5 reps.

EXPERIMENT RUN

`03-BKP1/results/2026.10.18.00{2,3,4}_threads_bs*_100000x100000.csv`.

ANALYSIS

T	BS 1024: GF/S, SPEEDUP / CAP	BS 1536	BS 3072
1	20.8, 1.00 / 1.0	21.0, 1.00 / 1.0	22.3, 1.00 / 1.0
8	155.6, 7.49 / 7.5	148.8, 7.10 / 7.2	142.2, 6.38 / 6.5

12	222.8, 10.72 / 10.9	212.0, 10.11 / 10.9	219.8, 9.86 / 10.9
16	**283.1, 13.62 / 14.0**	248.5, 11.85 / 13.0	205.2, 9.20 / 10.9

- The speedups follow the imbalance caps' staircase (bs = 3,072 is flat from 11 to 16 threads, exactly where the busiest thread's row count doesn't change), at 84–100% of the cap.
- Best: 283.1 GFLOP/s at bs = 1,024, 13.6×, 50.3% of 16-core TPP. Smaller blocks win from 4 threads up although bs = 3,072 is fastest on one core.
- No per-cluster ceiling of the kind the PA2 outer-loop reduction showed.

CONCLUSIONS

Supported: the static split of whole rows explains the shape and most of the loss. A further 2–16% at high thread counts, more for larger blocks, is unexplained here.

Experiment 10: one tile at a time, collapse(2) (BKP2) `parallel`

QUESTION

Making the unit of work a single tile removes the imbalance. Does every block size then scale linearly?

HYPOTHESIS

With 9,604 / 4,356 / 1,089 tiles, imbalance is negligible and the staircase disappears. bs = 3,072, fastest on one core, becomes the best at 16 threads (~330 GFLOP/s).

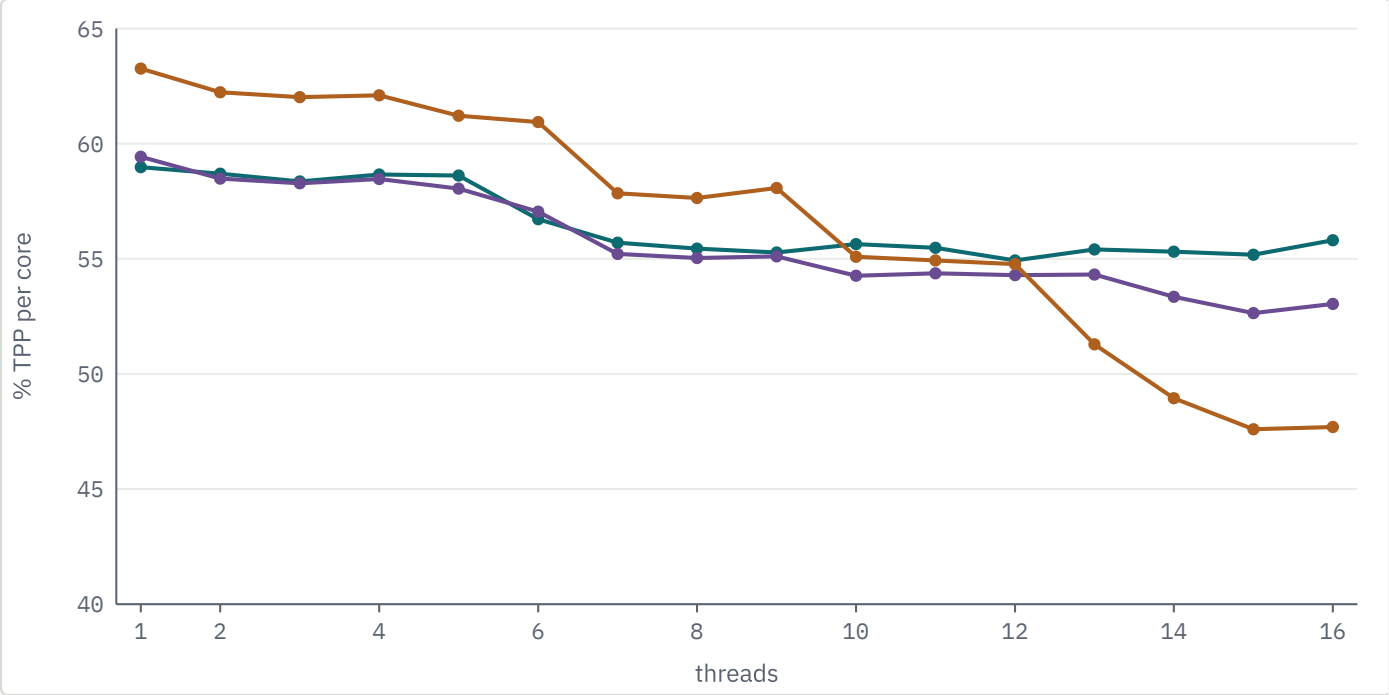
EXPERIMENT DESIGN

BKP1 plus `collapse(2)` over `ii` and `jj` (`iend` moved inside so the loops are perfectly nested). Same sweeps.

EXPERIMENT RUN

`03-BKP2/results/2026.10.18.00{2,3,4}_threads_bs*_100000x100000.csv`. Reps within 1–3.6%.

ANALYSIS



- The staircase is gone; best 314.3 GFLOP/s at bs = 1,024 (15.1×, 55.8%), +11% over BKP1; +31% for bs = 3,072.
- bs = 3,072 did not become the best; it is the slowest at 16 threads (268.6). Its per-thread rate falls steadily from ~62% to ~48% of TPP as threads are added, while bs = 1,024 levels off at 55–56% from 6 threads. Imbalance can't explain it (at most one extra tile per thread).

FOLLOW-UP: IS IT THE CLUSTER'S SHARED L2?

Three counter groups at bs = 1,024 and 3,072, 4 threads (one per cluster) and 16 (four per cluster), summed over threads, per vector FMA:

	1024, T4	1024, T16	3072, T4	3072, T16
cycles	1.702	1.822	1.596	2.064
L1 misses, % of loads	0.44%	0.76%	7.4%	14.3%
load stall cycles waiting on L2 hits	0.008	0.014	0.136	0.551
stall cycles per L2 request	0.40	0.59	0.46	1.76
memory / FP scheduler full, % of slots	13.6 / 37.0	12.6 / 34.1	20.9 / 35.6	38.2 / 18.5

For bs = 3,072, stalls on L2 hits account for ~88% of the extra 0.47 cycles per FMA at 16 threads: each L2 access gets ~4× slower when four threads share a cluster's L2, and L1

misses per FMA double. bs = 1,024 hardly misses L1, so the shared L2 barely touches it. (Counted 74–87% of the time; groups of 7–8 events were multiplexed.)

CONCLUSIONS

Partly supported: collapse removes the imbalance, but with four threads per cluster large tiles become L2-bound, which makes the smallest block tested the best. Why L1 misses per FMA double with more threads is not explained.

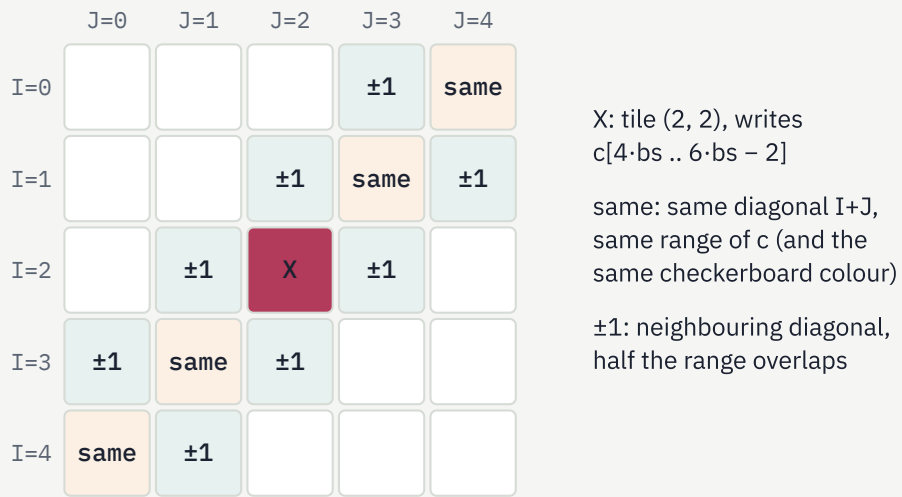
Experiment 11: the checkerboard, and which tiles really race (BKP3) `parallel` `diagnostic`

QUESTION

PA3.md's second approach drops the reduction: "a tile has a race only with its immediate four neighbors", so a checkerboard of tiles, run one colour at a time, should be race-free. Is it, for this tiling, and what does dropping the private copies buy?

HYPOTHESIS

Checked first by brute force (bs = 4, 5 × 5 tiles): tile `(I, J)` writes `c[(I+J)·bs .. (I+J+2)·bs − 2]`, so two tiles overlap exactly when their `I + J` differ by 0 or 1. All tiles on the same anti-diagonal write the same range of `c`, and they have the same checkerboard colour, so the literal checkerboard races (30 same-colour overlapping pairs in the 5 × 5 example). The parallel checking build should fail; the sequential one should pass.



That does not match PA3.md's "four neighbors" statement for this tiling (perhaps the lecture assumed a different index space; not resolved). It was built anyway, as a diagnostic, to see the race and its cost.

EXPERIMENT DESIGN

BKP2 without the reduction: two phases (colour 0, then 1), each a `parallel for collapse(2)` over the tile grid that skips the other colour; the implicit barrier separates the phases; threads add straight into the shared `c`. Timed with `sweep.sh --allow-gate-failure` (results under `results/diagnostic/`).

EXPERIMENT RUN

Gate, twice: sequential build passed; parallel build failed 11–12 of 42 oracle and 4 of 14 gold cases, every failing case with several tile rows (e.g. 20K × 20K random: ~31,000 of 40,001 outputs wrong, worst error 3–4·10^4 × the tolerance).

The first timing run was half of BKP2 *already at one thread*, where nothing can race (10.7 vs 20.8 GFLOP/s at bs = 1,024). The non-OpenMP build of the same source ran at full speed. `-fopt-info-vec` reported the fused loop "versioned for vectorization because of possible aliasing", and `perf annotate` showed a store of `c` after every FMA, 8 per 8 FMAs instead of one.

▶ Background: why OpenMP lost the restrict guarantee

Fix: the body of one tile moved into `bkq_tile`, a `noinline` function whose pointer parameters are all `restrict`-qualified. One store per 8 FMAs again, no aliasing versioning, 1-thread speed back to 20.7 GFLOP/s. The diagnostic timing was re-run on the fixed code (bs = 1,024 and 3,072, 5 reps; `03-BKP3/results/diagnostic/2026.10.18.00{3,4}_*`).

ANALYSIS

T	BKP3 BS 1024 (BKP2), RATIO	BKP3 BS 3072 (BKP2), RATIO
1	20.7 (20.8), 1.00	22.3 (22.3), 1.00
3	43.2 (61.6), 0.70	65.4 (65.5), 1.00
5	102.7 (103.2), 1.00	107.8 (107.7), 1.00
8	133.5 (156.1), 0.86	134.4 (162.3), 0.83
16	222.2 (314.3), 0.71	229.1 (268.6), 0.85

(Results numerically wrong; timing only.) BKP3 equals BKP2 at some thread counts and is 7–33% slower at others, slower at every count from 6 up, with rep spreads up to 12.5% (BKP2: ≤ 3.6%). Which thread counts lose is not explained; one reading (not measured) is that the static split decides whether same-diagonal tiles, which write the same cache lines of `c`, happen to run at the same time.

CONCLUSIONS

Refuted twice over: the literal checkerboard races for this tiling, and even ignoring correctness it is slower than the reduction once concurrently running tiles share ranges of `c`. The useful results are the overlap rule, which gives a race-free schedule directly, and the `restrict` fix, which any version without a reduction needs.

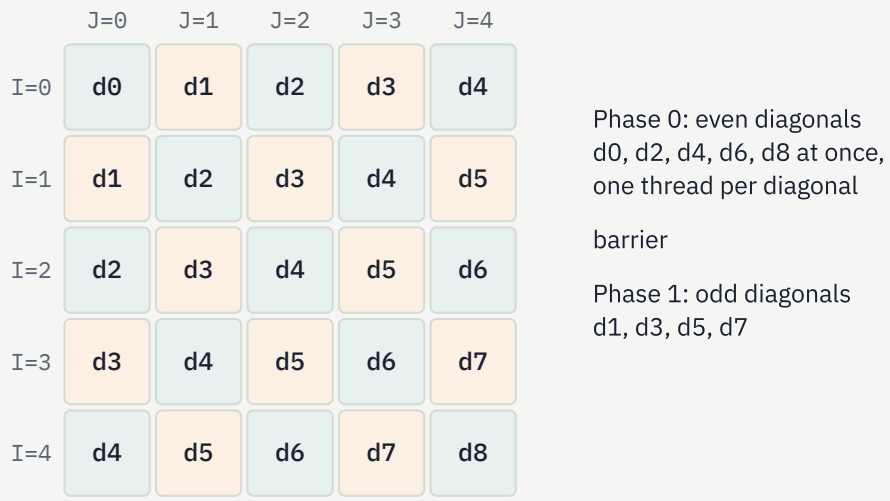
Experiment 12: diagonal parity, race-free without a reduction (BKP4) `parallel`

QUESTION

Tiles on diagonals `d` and `d + 2` never write the same element. Running whole diagonals of one parity at a time is race-free with no private copies. Is it faster than the reduction?

HYPOTHESIS

It passes the gate, matches BKP2 at one thread and matches or beats it at 16, since it skips zeroing and merging the private copies. Load imbalance should be small: a greedy simulation of dynamic scheduling in diagonal order (unit tile cost, edges ignored) gives 99.5% / 99.0% / 98.6% efficiency at 16 threads for bs = 1,024 / 1,536 / 3,072.



EXPERIMENT DESIGN

BKP3's restrict-qualified `bkq_tile`, and two phases: `#pragma omp parallel for schedule(dynamic, 1)` over the even diagonals, then the odd ones. One iteration is one whole diagonal, run in order by one thread, so no two threads ever hold tiles of the same diagonal; the barrier between phases keeps neighbouring diagonals apart. Same sweeps as BKP2.

EXPERIMENT RUN

Gate passed (three times). `03-BKP4/results/2026.10.18.00{2,3,4}_threads_bs*_100000x100000.csv`.

ANALYSIS

T	BS 1024 (BKP2), RATIO	BS 1536 (BKP2), RATIO	BS 3072 (BKP2), RATIO
1	20.7 (20.8) 1.00	21.0 (20.9) 1.00	22.3 (22.3) 1.00
8	164.6 (156.1) 1.05	164.7 (155.0) 1.06	174.1 (162.3) 1.07
12	246.7 (232.0) 1.06	238.5 (229.3) 1.04	224.3 (231.3) 0.97
16	**327.1** (314.3) 1.04	318.2 (298.7) 1.07	286.7 (268.6) 1.07

- Equal to BKP2 at 1–5 threads, 3–7% faster at almost every count from 6 up. Best 327.1 GFLOP/s, bs = 1,024, 15.8×, 58.1% of TPP.
- bs = 1,024 holds 57.3–58.9% of TPP per core at every thread count: near-linear scaling.
- bs = 3,072 still falls from 63% to 51%, the same shape as in BKP2. BKP4 has no private copies, so they are not the main cause of BKP2's L2 slowdown.
- Reps within 0.7% at bs = 1,024/1,536 except one slow rep at 12 threads (10.5% spread; median unaffected).

CONCLUSIONS

Supported: diagonal parity is race-free, costs nothing at one thread, and beats the reduction by a few percent. The phase barrier and the uneven diagonal lengths cost too little to see.

Experiment 13: the final kernel on 16 cores (BKP5) `parallel`

QUESTION

Does BKQ7's single-core gain carry into BKP4's schedule, and which block size is best once single-core speed no longer depends on it?

HYPOTHESIS

BKP5 keeps BKP4's near-linear scaling at BKQ7's per-core rate: ~22% over BKP4, ~400 GFLOP/s at 16 threads. The best block is set by the parallel side, probably 768–1,536.

EXPERIMENT DESIGN

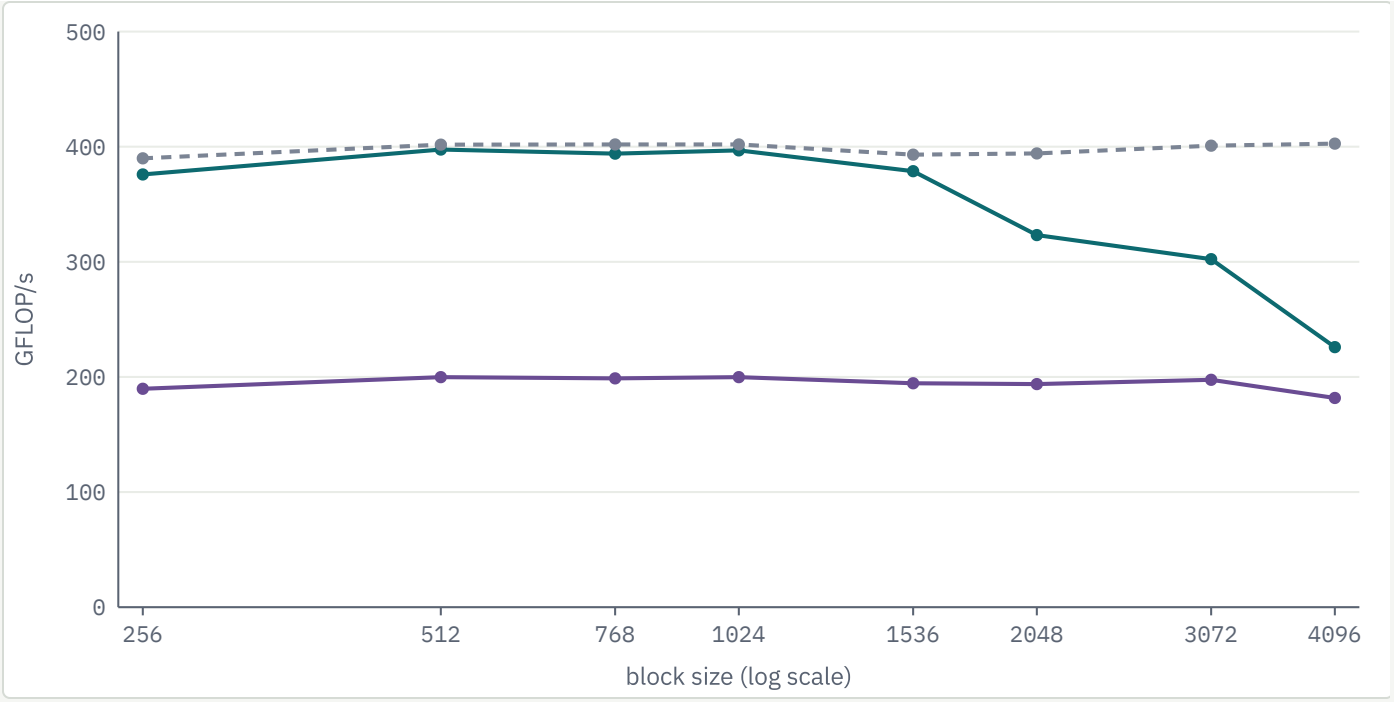
`bkq_tile` becomes BKQ7's tile body (per-tile-column zero-padded segments, fused loop over the whole trapezoid), in BKP4's diagonal-parity schedule.

> **A race BKQ7 didn't have.** BKQ7 rounds each group's loop up to a multiple of 8 with zero products, which writes up to 6 elements past the top of the tile's range of `c`. Adding 0 is still a read-modify-write, and those elements belong to diagonal `d + 2`, which runs at the same time. BKP5 keeps the padding inside `[ii + jj − 1, iend + jend − 2]`: above the group's range where there is room (every group but a tile's last), otherwise one element below (diagonal `d − 2` ends two below). Besides the gate, the write ranges were checked by enumeration with the same formulas, for 308 cases (gate sizes, 100K, edge shapes, 300 random shapes and block sizes): every diagonal's range ends below the start of diagonal `d + 2`.

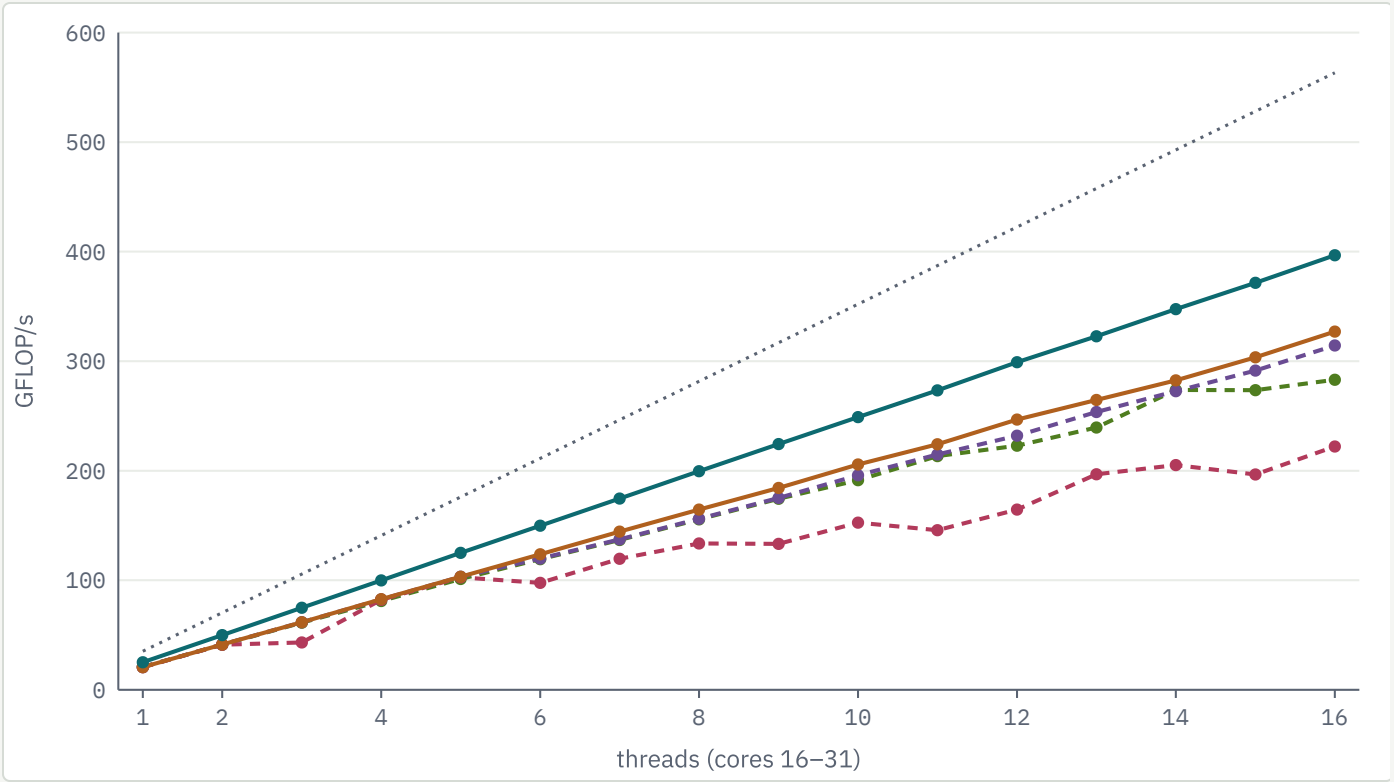
EXPERIMENT RUN

Gate passed. Threads 1–16 × blocks 256–4,096, 5 reps (`03-BKP5/results/2026.10.18.002_threads_x_bs_100000x100000.csv`), then a plain thread sweep at the best block (`.003`). One thread: 25.1 GFLOP/s, ~1% under BKQ7.

ANALYSIS



16 threads 8 threads 16 × the 1-thread rate



BKP1 rows + reduction, bs 1024 BKP2 tiles + reduction, bs 1024 BKP3 checkerboard (racy), bs 1024
BKP4 diagonal parity, bs 1024 BKP5 parity + BKQ7 kernel, bs 512 TPP (35.2 × threads)

T	BKP5 BS 512	BKP4 BS 1024	RATIO	BKP5 % TPP
1	25.1	20.7	1.21	71.3
4	100.0	82.7	1.21	71.0

8	199.6	164.6	1.21	70.9
12	299.0	246.7	1.21	70.8
16	**396.6**	327.1	1.21	70.4

- Blocks 512, 768 and 1,024 are within 1% of each other at every thread count and scale almost perfectly: bs = 512 goes 25.1 → 397.4 GFLOP/s (15.8×). 256 is ~5% lower throughout.
- Large blocks fall away above ~8 threads; bs = 4,096 drops sharply at 13 threads (265 → 217). At 100K it has only 25 diagonals per phase, so likely imbalance (not measured), on top of Experiment 10's L2 effect.
- BKP5 is 21–23% faster than BKP4 at every thread count; per-core rate 70.4–71.3% of TPP from 1 to 16 threads. Reps within 0.4% except one slow rep at 9 threads.

FOLLOW-UP: WHAT LIMITS BKP5?

Five counter groups, all counted 100% of the time, 100K, bs = 512 (user-mode counts, summed over threads; script `03-BKP5/perf_groups.sh`):

PER USEFUL VECTOR FMA	BKQ4 1536, 1 CORE	BKP5, T = 1	BKP5, T = 16
cycles (1.0 = TPP)	1.654	1.393	1.426
load µops	2.372	2.344	2.345
split loads	0.065	0.009	0.009
L1 misses, % of loads	9.3%	0.20%	0.24%
load buffer full, % of cycles	12.9%	0.8%	1.3%
memory scheduler full, % of slots	23.9%	2.0%	2.2%
FP scheduler full, % of slots	30.4%	**48.3%**	**47.3%**
retiring, % of slots	40.3%	44.7%	43.9%

- The memory side has dropped out. The per-tile-column segments also cured the old L1 misses (9.3% → 0.2% of loads).
- The FP scheduler is full 48% of slots: FMAs waiting for the previous FMA of their chain. At 1.39 cycles per FMA about 48 / (8 × 1.39) ≈ 4.3 chains must be in flight, ~35 waiting FMAs, which apparently fills it (the scheduler's size is not in the references; not measured).
- Two ceilings remain above it. The load port: 2.34 load µops per FMA on two 128-bit load pipes is a floor of ~1.17 cycles per FMA, ~85% of TPP. FMA throughput: 100%.

- Threading costs 2.4% (a little front-end and barrier waiting, 1.1% of slots).

CONCLUSIONS

Supported: 397 GFLOP/s, 70.4–70.6% of 16-core TPP, 21% above BKP4, with near-linear scaling. Almost all of the gap to TPP is inside the single-core kernel, now bound by the FMA dependency chains.

Summary

AT 100K × 100K	GFLOP/S	% TPP	OVER BKQ
OPQ (PA2, ≈ 27%)	≈ 9.4	≈ 27	0.7×
BKQ, plain blocking (1 core)	13.22	37.6	1.0×
BKQ4, aligned copies, 4,096-byte spacing (1 core)	22.23	63.2	1.7×
BKQ7, vector edges (1 core)	25.40	72.2	1.9×
BKP1, tile rows + reduction, 16 threads	283.1	50.3	21.4×
BKP2, tiles + reduction, 16 threads	314.3	55.8	23.8×
BKP4, diagonal parity, 16 threads	327.1	58.1	24.7×
BKP5, diagonal parity + BKQ7 kernel, 16 threads	**397.4**	**70.6**	**30.1×**

FINAL CONFIGURATIONS

- **BKQ7** (Part A): square tiles, R = 8 unroll-and-jam with the chain of 8 FMAs kept (`no-associative-math`), four shifted copies of `b` built as zero-padded 64-byte-aligned segments per tile column, spaced a multiple of 4,096 bytes, fused loop over each row group's whole trapezoid. 72.2% TPP on one E-core at bs = 768, 70–72% at any block from 256 to 4,096.
- **BKP5** (Part B): BKQ7's tile body in a `noinline`, `restrict`-qualified function; tiles grouped by anti-diagonal, even diagonals then odd, whole diagonals dealt with `schedule(dynamic, 1)`; zero padding kept inside each tile's own range of `c`. No reduction. 397 GFLOP/s at 16 threads, bs = 512 (512–1,024 equivalent).

THE MOST USEFUL THINGS LEARNED

1. Each single-core step removed one limit and exposed the next: loads per FMA (unroll-and-jam), compiler regrouping (keep the chain), split loads (aligned copies), placement of the

copies, scalar edge work, and now the FMA chain itself. Measuring before each step (the BKQ3 profile pointed at splits, not latency) avoided working on the wrong one.

2. Where the copies of `b` sit relative to each other mod 4,096 bytes changed the rate by up to 1.4× at the same block size; a spacing that is an exact multiple of 4,096 was best at every size.
3. The fixed scalar edge work per row group cost far more than its share of the products suggested. Doing it inside the vector loop, with zero-padded copies, gave +22% and made the block size irrelevant on one core.
4. For `(i, j)` tiles of this kernel, every tile on an anti-diagonal writes the same range of `c`: the races are along diagonals, not just with four neighbours. Diagonals of one parity are mutually race-free, and that schedule beat the array reduction by 3–7%.
5. Writes of zero are still writes: padding that never changes a value can still race if another thread owns the element.
6. OpenMP's outlining drops `restrict` on shared pointers, which halved the kernel in BKP3. Putting the tile body in a `restrict`-qualified function fixed it; the disassembly check (one store of `c` per 8 FMAs) caught it.

OPEN ITEMS

- The kernel is now bound by the FMA chains (FP scheduler full 48%). Nick's PA2 report used U = 3 interleaved accumulators for this; BKQ5's `#pragma GCC unroll` did not interleave and was 14–22% slower at large blocks, unexplained. Shorter chains (two partial sums per output) would change the summation order and how `BPM_ACC` is applied.
- Past ~85% of TPP the load port is the limit: fewer loads per FMA would need `b` vectors reused across rows within 16 registers.
- Unexplained: why BKQ7 gained twice the model's estimate; the ~2% dip at bs = 1,536–2,048 in BKQ7; w = 4 slower than w = 2; whether the placement effect is L1 sets or 4K aliasing; why BKP2's L1 misses per FMA double with threads; bs = 4,096's drop at 13 threads in BKP5.
- PA3.md's "four neighbors" race statement does not match this tiling; worth asking the instructor which index space it meant.
- The shared gate tune has no `block_width`; rectangular tiles were only checked with scratch builds. Adding it needs an approved change to `shared/include/kernel.h`.
- The copies of `b` are built serially inside the timed kernel; their cost at 16 threads was not measured.

NOTES ON METHOD

- `sweep.sh --record-seq` refuses `--tune`, so each recorded T(seq) is the untuned default (bs = 64). Thread-sweep speedups are against each sweep's own 1-thread time (`min` of the

two, as the harness does).

- `plot.py` overlays several CSVs only on a thread axis. The cross-variant single-core plot therefore uses fresh one-configuration runs of each variant at its best block, and the BKP5 comparison a separate bs = 512 thread sweep.
- Another user's rsync ran for much of the session and sometimes moved onto core 16. Rep spreads are reported with each experiment; the few large ones were single slow reps with the median unaffected.
- The BKQ5 control build, the rectangular-tile gate builds and the `schedule-insns` test were scratch builds outside the project; their timings are quick checks (3 reps), not sweep CSVs.
- Workbooks: each variant's `plots/` directory; cross-variant plots in `plots/`. The step-by-step log, including every command, is `record.md`.

Terms

"Glossary" is `reference/Glossary.md`; the Procinfo files are in `reference/` too. Terms marked "this report" are in neither.

TERM	WHAT IT MEANS HERE	READ MORE
RATES AND PEAKS		
flop, GFLOP/s	One floating-point multiply or add; billions per second. Each (i, j) pair is 2 flops.	Glossary: FLOP
TPP	Theoretical program peak, 35.2 GFLOP/s per E-core (2.2 GHz × 16 flops per cycle). % TPP = GFLOP/s ÷ (35.2 × threads).	Glossary: TPP. Procinfo1 § 2
speedup, T(seq)	Sequential time ÷ time with t threads; here against the same variant and block on one thread.	EXPERIMENTS.md
vector FMA	One 256-bit fused multiply-add: 8 multiply-adds, 16 flops.	Glossary: FMA, SIMD
THE MACHINE		
E-core, cluster	The 16 small cores (cpus 16–31), in 4 clusters of 4 sharing one L2 each.	Procinfo1 § 1
L1, L2	32 KB per E-core (8-way, 64 sets); 4 MB per cluster.	Procinfo1 § 4; /sys cache info

cache line, set	64-byte unit of memory traffic; the set is fixed by address bits 6–11, so addresses 4,096 bytes apart share a set.	Glossary
µop, load port	Internal steps of an instruction; a 256-bit load, store or FMA is two. Two 128-bit load pipes: one 256-bit load per cycle.	Procinfo2 § 1.A; Procinfo3 § 2.D
pipeline slot	One chance per cycle to accept one µop; 5 per cycle on an E-core.	Procinfo3 § 1.B
latency, dependency chain	An FMA's result takes 6 cycles; a chain is a run of FMAs each needing the previous result.	Procinfo2 § 2.A
split load	A load that straddles two cache lines.	Glossary: Cache-line split
load buffer	The queue that tracks loads in flight.	Glossary: Load queue
FP / memory scheduler	Where accepted µops wait for inputs and a port. Counters `topdown_be_bound.non_mem_scheduler` / `.mem_scheduler`.	this report
4K aliasing	A load held up because its address matches an older store's in the low 12 bits. Counter `ld_blocks.address_alias`.	this report

THE KERNEL

tile, block_size	A `bs × bs` piece of the (i, j) rectangle, finished before the next.	this report
unroll-and-jam, fused loop	8 rows handled together in one loop over t, all adding into `c[i0+t]`.	Experiment 2
ragged ends	The products of a row group outside the fused loop's range; scalar until BKQ7.	Experiments 2, 7, 8
shifted copies, segment	Copies of `b` offset by 0–3 floats so every row's loads are aligned; in BKQ7/BKP5 one zero-padded segment per tile column.	Experiments 4, 8
reassociation	`-ffast-math` lets the compiler regroup additions; `no-associative-math` on the kernel keeps the 8-FMA chain.	Experiment 3

gate (oracle, gold)	The shared correctness check, with Kahan-compensated accumulation in the checking build.	READ_ME_FIRST.md; DESIGN.md

PARALLELIZING

race	Two threads update the same `c[k]` without coordination and an update is lost.	this report
array reduction	Each thread adds into a private copy of `c`; the copies are summed at the end.	Experiment 9
collapse(2)	OpenMP treats the two tile loops as one, so threads are dealt single tiles.	Experiment 10
anti-diagonal, d	Tiles with the same I + J; they write the same range of `c`.	Experiment 11
diagonal parity	Run all even diagonals, barrier, then all odd ones; each diagonal by one thread.	Experiment 12
schedule(static / dynamic, 1)	Fixed contiguous chunks per thread / one iteration at a time to whichever thread is free.	OpenMP
outlining, restrict	GCC moves a parallel loop's body into a separate function; `restrict` promises a pointer doesn't alias others, and is lost on shared pointers there.	Experiment 11